`Form` object can save you from writing a lot of HTML, right up until you hit its limits, and then you're back to writing all the HTML. If your site uses a framework like Bootstrap, which requires wrapping `<div>` tags in forms for styling, you may have to write the HTML by hand.

Keeping the limitations in mind, let's start by building a simple form, so you can see what it can and can't do. The `Bands` app is getting a little full, so for this chapter, create a new app, called `Content`, using the `startapp` management command. Don't forget to add it to `INSTALLED_APPS` in settings.py as well. See Chapter 2 if you need a refresher on how to do this.

Inside your newly created RiffMates/content directory, create a file called forms.py to build your first `Form` class. *Form objects* are quite similar to `Model` objects: they contain fields that specify areas of input in the form. Like with a model, you inherit from a class and declare attributes to build the form. The class you inherit from is `django.forms.Form`. Inside the form class, you add attributes for each field you want, with a variety of fields provided in the `forms` module.

Your first form is for collecting comments from your site's visitors. It has a name field and a comment area. The corresponding HTML contains an `<input>` tag for the name and a `<textarea>` tag for the comment. As both the name and comment fields collect text information, they get declared with a `forms.CharField()`. By default, this field uses an `<input>` tag. To use a `<textarea>` instead, you add a `widget` argument to the field's constructor.

Every field type supports changing the widget used to render it, and every field type has a default. If you're happy with the default, you don't need to specify a widget. To use a `<textarea>` tag, you pass a `forms.Textarea` widget to the field declaration. You can control the attributes of a rendered widget by passing an `attrs` dictionary to the widget class. Any key–value pairs in the dictionary get rendered as attributes in the corresponding HTML tag. You use this feature to specify the `rows` and `cols` values for a `<textarea>` tag. Create a comment form by adding the code in the following listing to your newly created file.

Listing 7.1 A form class for accepting comments

```
# RiffMates/content/forms.py
from django import forms

class CommentForm(forms.Form):
    name = forms.CharField()
    comment = forms.CharField(
        widget=forms.Textarea(
            attrs={"rows": "6", "cols": "50"}
        )
    )
```

The forms module contains the form base class, fields, and widgets.

Inherit from forms.Form to create a Form object.

forms.CharField is used to collect text and uses an <input> tag by default.

Specify the widget attribute to change what tag gets used to collect data.

You can pass additional attributes to the tag using the attrs dictionary argument in the widget object constructor.